# 数据库设计与开发
Database Design & Development

计算机学院　单纯
sherryshan@bit.edu.cn
2024年3月

# 数据库设计
Database Design

计算机学院　单纯
sherryshan@bit.edu.cn
2024年4月

## 内容概览

- 5.1 数据库设计概述
- 5.2 需求分析
- 5.3 概念结构设计
- 5.4 逻辑结构设计
- 5.5 数据库的物理设计
- 5.6 数据库实施和维护
- 5.7 IDEF1X数据建模
- 5.8 小结

## 数据库设计（1）

- 数据库设计
  - 数据库设计是指对于一个给定的应用环境，构造（设计）优化的数据库逻辑模式和物理结构，并据此建立数据库及其应用系统，使之能够有效地存储和管理数据，满足各种用户的应用需求，包括信息管理要求和数据操作要求
  - 目标：为用户和各种应用系统提供一个信息基础设施和高效率的运行环境

## 数据库设计（2）

- 数据库设计人员应该具备的技术和知识
  - 数据库的基本知识和数据库设计技术
  - 计算机科学的基础知识和程序设计的方法和技巧
  - 软件工程的原理和方法
  - 应用领域的知识

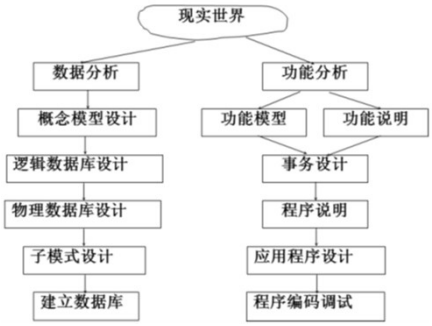
## 5.1 数据库设计概述

- 5.1.1 数据库设计的特点
- 5.1.2 数据库设计方法
- 5.1.3 数据库设计的基本步骤
- 5.1.4 数据库设计过程中的各级模式

## 5.1.1 数据库设计的特点（1）

- 数据库建设的基本规律
  - 三分技术，七分管理，十二分基础数据
    - 管理
      - 数据库建设项目管理
      - 企业（即应用部门）的业务管理
    - 基础数据
      - 收集、入库
      - 更新新的数据
- 结构（数据）设计和行为（处理）设计相结合
  - 将数据库结构设计和数据处理设计密切结合

## 5.1.1 数据库设计的特点（2）




## 5.1.2 数据库设计方法（1）

- 手工试凑法
- 规范设计法
- Barker设计方法

## 手工试凑法（1）

- 手工试凑法（直接设计法）：根据应用的数据要求与处理要求，直接设计数据库的结构
- 设计质量与设计人员的经验和水平有直接关系
- 数据库运行一段时间后常常不同程度地发现各种问题，增加了维护代价

## 手工试凑法（2）

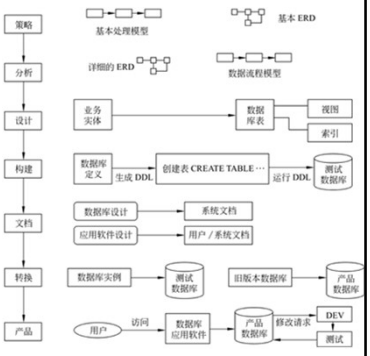
- 缺点
  - 数据间关系复杂，使用要求各式各样，很难仅凭经验进行设计
  - 把数据的逻辑结构，物理结构、处理要求等一起考虑，很难对模式进行评价和优化。用户需求一旦发生变化，数据结构很难随之发生变化
  - 数据库设计与具体的DBMS紧密结合，移植困难
  - 缺乏文档资料，难于与用户交流，对设计难于评审
  - 难以由多个人合作进行设计

## 规范设计法

- 规范设计法：运用软件工程的思想和方法，把整个设计过程划分为若干阶段，把复杂的大问题分为若干相对简单的小问题，每个阶段只解决整个设计中的部分问题
- 规范设计法是迭代和逐步求精的过程，每一过程完成时要进行设计分析，产生各种设计文档，并组织评审和用户交流，如不满足要求则进行修改

## Barker设计方法

- Barker方法是以Oracle公司的董事Richard Barker先生的名字命名的。右图所示为Barker数据库设计方法的过程




## 5.1.2 数据库设计方法（2）

- 新奥尔良（New Orleans）方法
  - 将数据库设计分为若干阶段和步骤
- 基于E-R模型的数据库设计方法
  - 概念设计阶段广泛采用
- 3NF（第三范式）的设计方法
  - 逻辑阶段可采用的有效方法
- ODL（Object Definition Language）方法
  - 面向对象的数据库设计方法

## 5.1.2 数据库设计方法（3）

- 计算机辅助设计
  - ORACLE Designer
  - SAP PowerDesigner
  - CA ERwin Data Modeler

## 5.1.3 数据库设计的基本步骤（1）

- 按照规范设计方法，根据软件工程的思想，数据库的设计可以分为以下六个阶段
  - 需求分析
  - 概念结构设计
  - 逻辑结构设计
  - 物理结构设计
  - 数据库实施
  - 数据库运行和维护

## 5.1.3 数据库设计的基本步骤（2）

- 需求分析和概念设计独立于任何数据库管理系统
- 逻辑设计和物理设计与选用的DBMS密切相关

## 5.1.3 数据库设计的基本步骤（3）

- 一、数据库设计的准备工作
- 二、数据库设计的过程（六个阶段）
- 三、数据库设计的特点

# 一、数据库设计的准备工作

- 选定参加设计的人
  1. 系统分析人员、数据库设计人员
     - 自始至终参与数据库设计
  2. 用户和数据库管理员
     - 主要参加需求分析和数据库的运行维护
  3. 应用开发人员（程序员和操作员）
     - 在系统实施阶段参与进来，负责编制程序和准备软硬件环境

# 二、数据库设计的过程（六个阶段）（1）

1. 需求分析阶段
   - 准确了解与分析用户需求（包括数据与处理）
   - 最困难、最耗费时间的一步
2. 概念结构设计阶段
   - 整个数据库设计的关键
   - 通过对用户需求进行综合、归纳与抽象，形成一个独立于具体DBMS的概念模型

# 二、数据库设计的过程（六个阶段）（2）

3. 逻辑结构设计阶段
   - 将概念结构转换为某个DBMS所支持的数据模型
   - 对其进行优化
4. 数据库物理设计阶段
   - 为逻辑数据模型选取一个最适合应用环境的物理结构（包括存储结构和存取方法）

# 二、数据库设计的过程（六个阶段）（3）

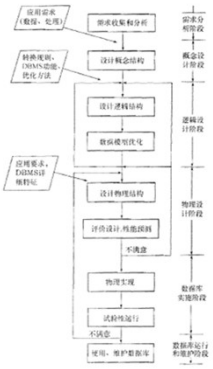
5. 数据库实施阶段
   - 运用DBMS提供的数据库语言（如SQL）及宿主语言，根据逻辑设计和物理设计的结果
     - 建立数据库
     - 编制与调试应用程序
     - 组织数据入库
     - 进行试运行

# 二、数据库设计的过程（六个阶段）（4）

6. 数据库运行和维护阶段
   - 数据库应用系统经过试运行后即可投入正式运行
   - 在数据库系统运行过程中必须不断地对其进行评价、调整与修改

# 二、数据库设计的过程（六个阶段）（5）

- 设计一个完善的数据库应用系统往往是上述六个阶段的不断反复

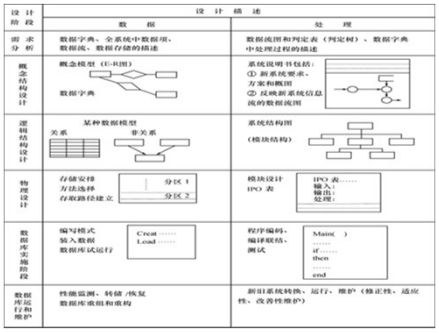


## 三、数据库设计的特点（1）

- 把数据库设计和对数据库中数据处理的设计紧密结合起来
- 将这两个方面的需求分析、抽象、设计、实现在各个阶段同时进行，相互参照，相互补充，以完善两方面的设计

## 三、数据库设计的特点（2）

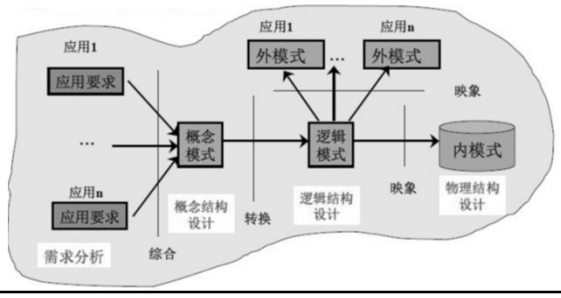
- 设计过程各个阶段的设计描述



| 设计阶段 | 设计描述 | |
|---|---|---|
| | 数据 | 处理 |
| 需求分析 | 数据字典、全系统中数据项、数据流、数据存储的描述 | 数据流图和判定表（判定树）、数据字典中处理过程的描述 |
| 概念结构设计 | 概念模型（E-R图）<br>数据字典 | 系统说明书包括：<br>① 新系统要求、方案和概图<br>② 反映新系统信息流的数据流图 |
| 逻辑结构设计 | 某种数据模型<br>关系　非关系 | 系统结构图<br>（模块结构） |
| 物理设计 | 存储安排<br>方法选择<br>存取路径建立<br>分区 1<br>分区 2 | 模块设计<br>IPO 表<br>IPO 表……<br>输入：<br>输出：<br>处理： |
| 数据库实施阶段 | 编写模式<br>装入数据<br>数据库试运行<br>Creat ……<br>Load …… | 程序编码、编译联结、测试<br>Main( )<br>……<br>if ……<br>then<br>……<br>end |
| 数据库运行和维护 | 性能监测、转储/恢复<br>数据库重组和重构 | 新旧系统转换、运行、维护（修正性、适应性、改善性维护） |

## 5.1.4 数据库设计过程中的各级模式

- 数据库设计不同阶段形成的数据库各级模式




## 5.2 需求分析

- 5.2.1 需求分析的任务
- 5.2.2 需求分析的方法
- 5.2.3 数据字典
- 5.2.4 需求分析小结

## 5.2.1 需求分析的任务

- 需求分析的任务
- 需求分析的重点
- 需求分析的难点

## 需求分析的任务

- 详细调查现实世界要处理的对象（组织、部门、企业等）
- 充分了解原系统（手工系统或计算机系统）
- 明确用户的各种需求
- 确定新系统的功能
- 充分考虑今后可能的扩充和改变

## 需求分析的重点

- 调查的重点是“数据”和“处理”，获得用户对数据库要求
  - 信息要求
  - 处理要求
  - 安全性与完整性要求

## 需求分析的难点

- 确定用户最终需求
  - 用户缺少计算机知识
  - 设计人员缺少用户的专业知识
- 解决方法
  - 设计人员必须不断深入地与用户进行交流

## 5.2.2 需求分析的方法

- 调查需求
- 达成共识
- 分析表达需求

## 调查用户需求的具体步骤

- 调查组织机构情况
- 调查各部门的业务活动情况
- 在熟悉业务活动的基础上，协助用户明确对新系统的各种要求
- 确定新系统的边界

## 常用调查方法

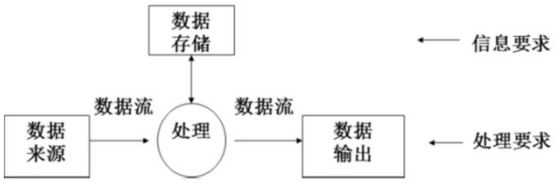
- 跟班作业
- 开调查会
- 请专人介绍
- 询问
- 设计调查表请用户填写
- 查阅记录

## 进一步分析和表达用户需求（1）

- 结构化分析方法（Structured Analysis，简称SA方法）
  - 从最上层的系统组织机构入手
  - 自顶向下、逐层分解分析系统

## 进一步分析和表达用户需求（2）

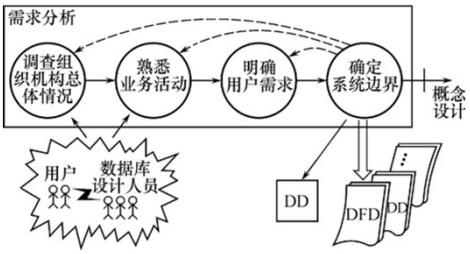
1. 首先把任何一个系统都抽象为




## 进一步分析和表达用户需求（3）

2. 分解处理功能和数据
   - （1）分解处理功能
     - 将处理功能的具体内容分解为若干子功能
   - （2）分解数据
     - 处理功能逐步分解同时，逐级分解所用数据，形成若干层次的数据流图
   - （3）表达方法
     - 处理逻辑：用判定表或判定树来描述
     - 数据：用数据字典来描述
3. 将分析结果再次提交给用户，征得用户的认可

## 需求分析过程




## 5.2.3 数据字典（1）

- 数据字典的用途
  - 进行详细的数据收集和数据分析所获得的主要结果
- 数据字典的内容
  - 数据项
  - 数据结构
  - 数据流
  - 数据存储
  - 处理过程

## 数据项

- 数据项是不可再分的数据单位
- 对数据项的描述
  - 数据项描述＝｛数据项名，数据项含义说明，别名，数据类型，长度，取值范围，取值含义，与其他数据项的逻辑关系，数据项之间的联系｝

## 数据结构

- 数据结构反映了数据之间的组合关系
- 一个数据结构可以由若干个数据项组成，也可以由若干个数据结构组成，或由若干个数据项和数据结构混合组成
- 对数据结构的描述
  - 数据结构描述＝｛数据结构名，含义说明，组成:｛数据项或数据结构｝｝

## 数据流

- 数据流是数据结构在系统内传输的路径
- 对数据流的描述
  - 数据流描述＝｛ 数据流名，说明，数据流来源，数据流去向，组成:｛数据结构｝，平均流量，高峰期流量｝

## 数据存储

- 数据存储是数据结构停留或保存的地方，也是数据流的来源和去向之一
- 对数据存储的描述
  - 数据存储描述＝｛数据存储名，说明，编号，输入的数据流，输出的数据流，组成:｛数据结构｝，数据量，存取频度，存取方式｝

## 处理过程

- 具体处理逻辑一般用判定表或判定树来描述
- 处理过程说明性信息的描述
  - 处理过程描述＝｛处理过程名，说明，输入:｛数据流｝， 输出:｛数据流｝，处理:｛简要说明｝｝

## 数据字典举例（1）

- 例：学生学籍管理子系统的数据字典
  - 数据项，以“学号”为例
    - 数据项： 学号
    - 含义说明：唯一标识每个学生
    - 别名： 学生编号
    - 类型： 字符型
    - 长度： 8
    - 取值范围：00000000至99999999
    - 取值含义：前两位表示该学生所在年级，后六位按顺序编号
    - 与其他数据项的逻辑关系：

## 数据字典举例（2）

- 例：学生学籍管理子系统的数据字典（续）
  - 数据结构，以“学生”为例，“学生”是该系统中的一个核心数据结构
    - 数据结构： 学生
    - 含义说明： 是学籍管理子系统的主体数据结构，定义了一个学生的有关信息
    - 组成： 学号，姓名，性别，年龄，所在系，年级

## 数据字典举例（3）

- 例：学生学籍管理子系统的数据字典（续）
  - 数据流，“体检结果”可如下描述
    - 数据流： 体检结果
    - 说明： 学生参加体格检查的最终结果
    - 数据流来源：体检
    - 数据流去向：批准
    - 组成： ……
    - 平均流量： ……
    - 高峰期流量：……

## 数据字典举例（4）

- 例：学生学籍管理子系统的数据字典（续）
  - 数据存储，“学生登记表”可如下描述
    - 数据存储： 学生登记表
    - 说明： 记录学生的基本情况
    - 流入数据流：……
    - 流出数据流：……
    - 组成： ……
    - 数据量： 每年3000张
    - 存取方式： 随机存取

## 数据字典举例（5）

- 例：学生学籍管理子系统的数据字典（续）
  - 处理过程“分配宿舍”可如下描述
    - 处理过程：分配宿舍
    - 说明： 为所有新生分配学生宿舍
    - 输入： 学生，宿舍
    - 输出： 宿舍安排
    - 处理： 在新生报到后，为所有新生分配学生宿舍。要求同一间宿舍只能安排同一性别的学生，同一个学生只能安排在一个宿舍中。每个学生的居住面积不小于3平方米。安排新生宿舍其处理时间应不超过15分钟

## 5.2.3 数据字典（2）

- 数据字典是关于数据库中数据的描述，是元数据，而不是数据本身
- 数据字典在需求分析阶段建立，在数据库设计过程中不断修改、充实、完善

## 5.2.4 需求分析小结

- 设计人员应充分考虑到可能的扩充和改变，使设计易于更改，系统易于扩充
- 必须强调用户的参与

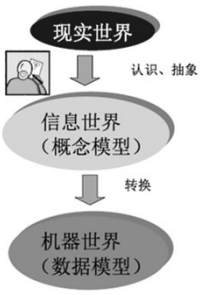
## 5.3 概念结构设计

- 5.3.1 概念结构
- 5.3.2 概念结构设计的方法与步骤
- 5.3.3 数据抽象与局部视图设计
- 5.3.4 视图的集成
- 5.3.5 概念结构设计小结

## 5.3.1 概念结构（1）

- 什么是概念结构设计
  - 将需求分析得到的用户需求抽象为信息结构即概念模型的过程就是概念结构设计
  - 概念结构是各种数据模型的共同基础，它比数据模型更独立于机器、更抽象，从而更加稳定
  - 概念结构设计是整个数据库设计的关键

## 5.3.1 概念结构（2）




## 5.3.1 概念结构（3）

- 概念结构设计的特点
  - 能真实、充分地反映现实世界
  - 易于理解
  - 易于更改
  - 易于向关系、网状、层次等各种数据模型转换

## 5.3.1 概念结构（4）

- 描述概念模型的工具
  - E-R模型

## 基于E-R模型的数据库设计方法（1）

- Peter Pin-Chen在1976年提出E-R法（Entity-Relation Approach，实体联系方法）也称为E-R模型
- E-R法的思想是用E-R图描述现实世界的信息，这种信息结构称为概念结构，然后根据具体系统的要求将概念结构转换成特定系统所能接受的逻辑结构（层次、网状、关系）

## 基于E-R模型的数据库设计方法（2）

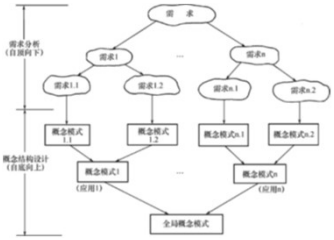
- E-R法由两部分组成
  - 用E-R图描述现实世界
  - 将E-R图转换成相应的数据模型

## 5.3.2 概念结构设计的方法与步骤（1）

- 设计概念结构通常有四类方法
  - 自顶向下：首先定义全局概念结构的框架，然后逐步细化
  - 自底向上：首先定义各局部应用的概念结构，然后将其集成起来
  - 逐步扩张：首先核心概念结构，然后向外扩充
  - 混合策略：自顶向下设计全局概念框架，自底向上设计各局部概念结构

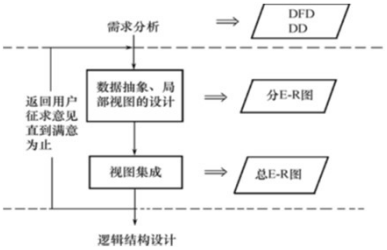
## 5.3.2 概念结构设计的方法与步骤（2）

- 常用策略
  - 自顶向下地进行需求分析
  - 自底向上地设计概念结构




## 5.3.2 概念结构设计的方法与步骤（3）

- 自底向上设计概念结构的步骤
  - 第1步：抽象数据并设计局部视图
  - 第2步：集成局部视图，得到全局概念结构




## 5.3.3 数据抽象与局部视图设计

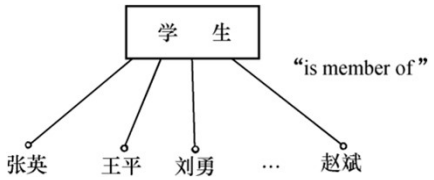
- 数据抽象
- 局部视图设计

## 数据抽象（1）

- 抽象是对实际的人、物、事和概念中抽取所关心的共同特性，忽略非本质的细节，并把这些特性用各种概念精确地加以描述
  - 概念结构是对现实世界的一种抽象

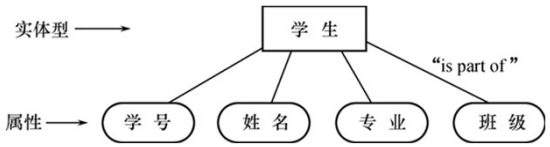
## 数据抽象（2）

- 三种常用抽象
  1. 分类（Classification）
     - 定义某一类概念作为现实世界中一组对象的类型
     - 抽象了对象值和型之间的“is member of”的语义



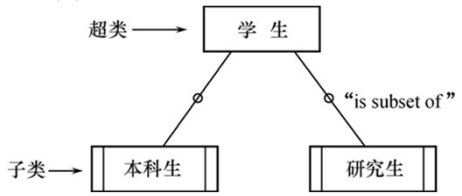

## 数据抽象（3）

- 三种常用抽象（续）
  2. 聚集（Aggregation）
     - 定义某一类型的组成成分
     - 抽象了对象内部类型和成分之间“is part of”的语义



# 数据抽象（4）

- 三种常用抽象（续）
  3. 概括（Generalization）
    - 定义类型之间的一种子集联系
    - 抽象了类型之间的“is subset of”的语义
    - 继承性

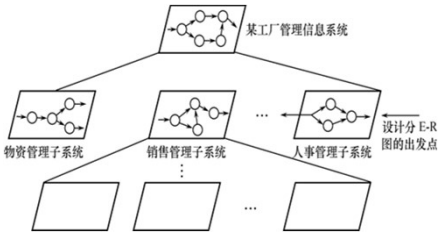



# 局部视图设计

- 设计分E-R图的步骤
  1. 选择局部应用
  2. 逐一设计分E-R图

# 1. 选择局部应用

- 在多层的数据流图中选择一个适当层次的数据流图，作为设计分E-R图的出发点
- 通常以中层数据流图作为设计分E-R图的依据




# 2. 逐一设计分E-R图（1）

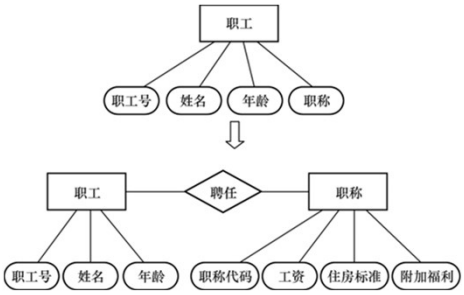
- 任务
  - 将各局部应用涉及的数据分别从数据字典中抽取出来
  - 参照数据流图，标定各局部应用中的实体、实体的属性、标识实体的键
  - 确定实体之间的联系及其类型（1:1，1:n，m:n）

# 2. 逐一设计分E-R图（2）

- 两条准则
  - 属性不能再具有需要描述的性质。即属性必须是不可分的数据项，不能再由另一些属性组成
  - 属性不能与其他实体具有联系。联系只发生在实体之间

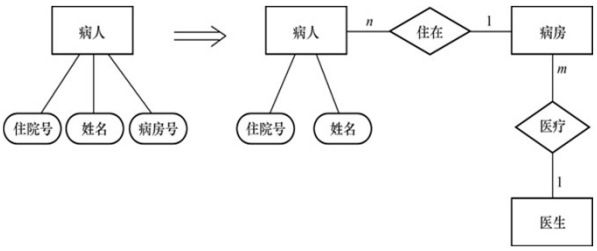
# 2. 逐一设计分E-R图（3）

- 职称作为一个实体



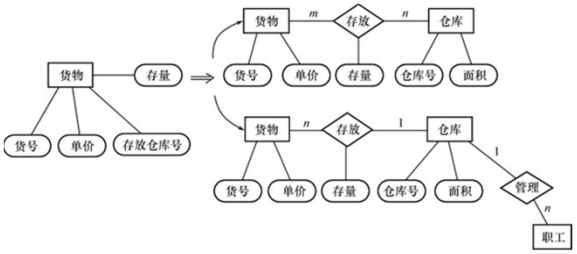
## 2. 逐一设计分E-R图（4）

- 病房作为一个实体




## 2. 逐一设计分E-R图（5）

- 仓库作为一个实体

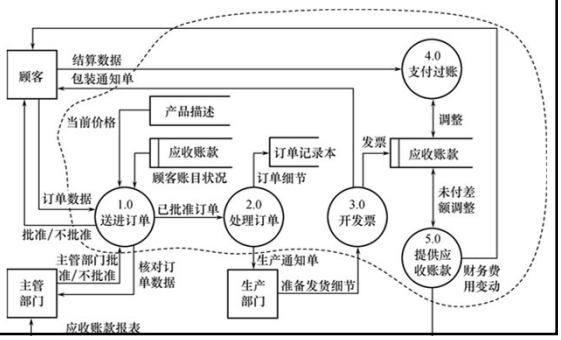



## 2. 逐一设计分E-R图（6）

- [实例] 销售管理子系统分E-R图的设计
  - 销售管理子系统的主要功能
    - 处理顾客和销售员送来的订单
    - 工厂是根据订货安排生产的
    - 交出货物同时开出发票
    - 收到顾客付款后，根据发票存根和信贷情况进行应收款处理

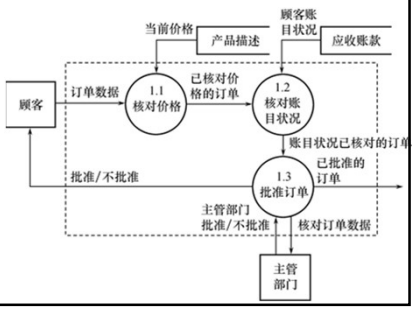
## 2. 逐一设计分E-R图（7）

- 第一层数据流图，虚线部分划出了系统边界



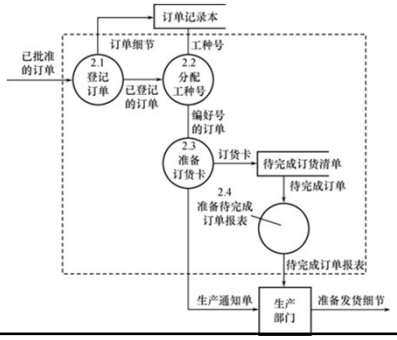

## 2. 逐一设计分E-R图（8）

- 系统功能分为4个子系统，下面四个图是第二层数据流图
  - 接收订单



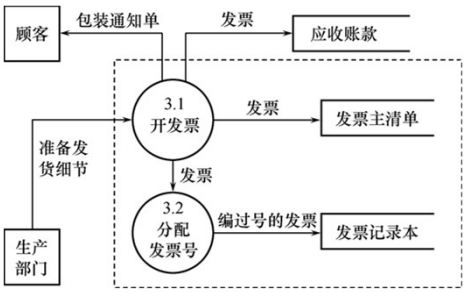

## 2. 逐一设计分E-R图（9）

- 第二层数据流图（续）
  - 处理订单



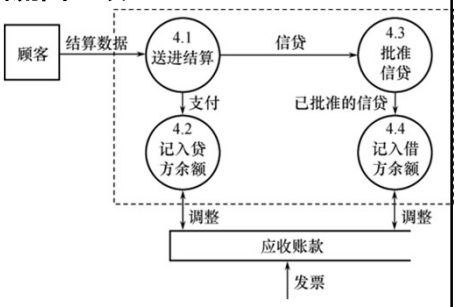
## 2. 逐一设计分E-R图（10）

- 第二层数据流图（续）
  - 开发票



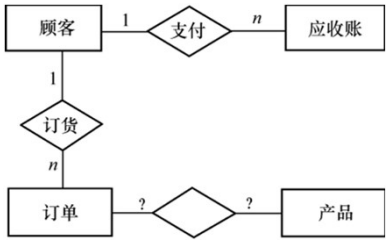

## 2. 逐一设计分E-R图（11）

- 第二层数据流图（续）
  - 支付过账




## 2. 逐一设计分E-R图（12）

- 分E-R图的框架

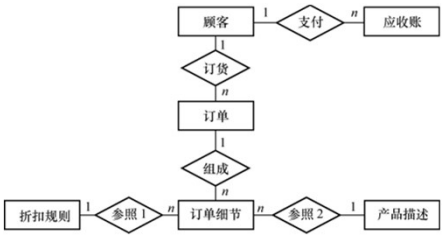



## 2. 逐一设计分E-R图（13）

- 参照第二层数据流图和数据字典，遵循两个准则，进行如下调整
  - 订单与订单细节是1：n的联系
  - 原订单和产品的联系实际上是订单细节和产品的联系
  - “发票主清单”是一个数据存储，不必作为实体加入分E-R图
  - 工厂对大宗订货给予优惠

## 2. 逐一设计分E-R图（14）

- 销售管理子系统的分E-R图




## 2. 逐一设计分E-R图（15）

- 对每个实体定义属性
  - 顾客：{顾客号，顾客名，地址，电话，信贷状况，账目余额}
  - 订单：{订单号，顾客号，订货项数，订货日期，交货日期，工种号，生产地点}
  - 订单细则：{订单号，细则号，零件号，订货数，金额}

## 2. 逐一设计分E-R图（16）

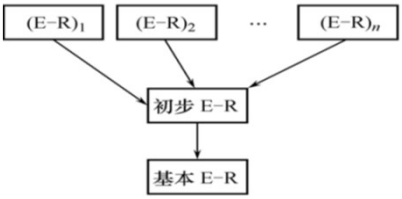
- 对每个实体定义属性（续）
  - 应收账款：{顾客号，订单号，发票号，应收金额，支付日期，支付金额，当前余额，货款限额}
  - 产品描述：{产品号，产品名，单价，重量}
  - 折扣规则：{产品号，订货量，折扣}

## 5.3.4 视图的集成（1）

- 各个局部视图即分E-R图建立好后，还需要对它们进行合并，集成为一个整体的数据概念结构即总E-R图

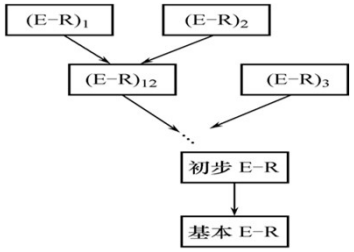
## 5.3.4 视图的集成（2）

- 视图集成的两种方式
  - 多个分E-R图一次集成
    - 一次集成多个分E-R图
    - 通常用于局部视图比较简单时




## 5.3.4 视图的集成（3）

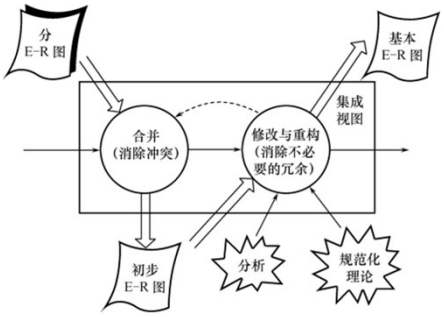
- 视图集成的两种方式（续）
  - 逐步集成
    - 用累加的方式一次集成两个分E-R图




## 5.3.4 视图的集成（4）

- 集成局部E-R图的步骤
  1. 合并
  2. 修改与重构

## 5.3.4 视图的集成（5）



## 合并分E-R图，生成初步E-R图

- 各分E-R图存在冲突
  - 各个分E-R图之间必定会存在许多不一致的地方
- 合并分E-R图的主要工作与关键
  - 合理消除各分E-R图的冲突
- 冲突的种类
  - 属性冲突
  - 命名冲突
  - 结构冲突

## 1. 属性冲突

- 两类属性冲突
  - 属性域冲突
    - 属性值的类型
    - 取值范围
    - 取值集合不同
  - 属性取值单位冲突

## 2. 命名冲突

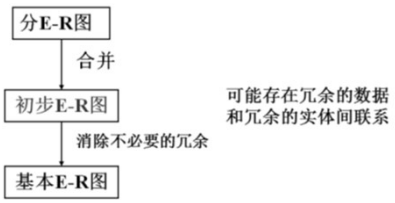
- 两类命名冲突
  - 同名异义：不同意义的对象在不同的局部应用中具有相同的名字
  - 异名同义（一义多名）：同一意义的对象在不同的局部应用中具有不同的名字

## 3. 结构冲突

- 三类结构冲突
  - 同一对象在不同应用中具有不同的抽象
  - 同一实体在不同分E-R图中所包含的属性个数和属性排列次序不完全相同
  - 实体之间的联系在不同局部视图中呈现不同的类型

## 消除不必要的冗余，设计基本E-R图（1）

- 基本任务
  - 消除不必要的冗余，设计生成基本E-R图




## 消除不必要的冗余，设计基本E-R图（2）

- 冗余
- 消除冗余的方法

## 冗余

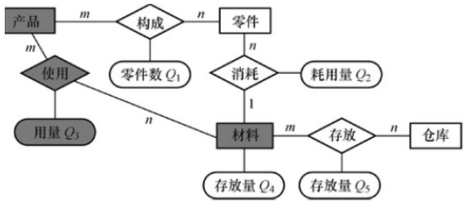
- 冗余的数据是指可由基本数据导出的数据
- 冗余的联系是指可由其他联系导出的联系
- 冗余数据和冗余联系容易破坏数据库的完整性，给数据库维护增加困难
- 消除不必要的冗余后的初步E-R图称为基本E-R图

## 消除冗余的方法（1）

- 分析方法
  - 以数据字典和数据流图为依据
  - 根据数据字典中关于数据项之间的逻辑关系

## 消除冗余的方法（2）

- 消除冗余




## 消除冗余的方法（3）

- 效率VS冗余信息
  - 需要根据用户的整体需求来确定
- 若人为地保留了一些冗余数据，则应把数据字典中数据关联的说明作为完整性约束条件
  - Q4=∑Q5
  - 一旦Q5修改后就应当触发完整性检查，对Q4进行修改

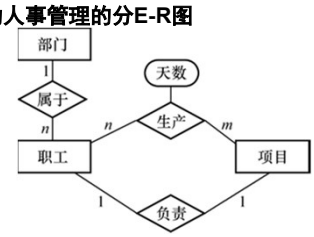
## 消除冗余的方法（4）

- 规范化理论
  - 函数依赖的概念提供了消除冗余联系的形式化工具

## 消除冗余的方法（5）

- 方法

  1. 确定分E-R图实体之间的数据依赖，并用实体键之间的函数依赖表示
     - 劳动人事管理的分E-R图



## 消除冗余的方法（6）

– 上图中
- 部门和职工之间一对多的联系可表示为
  - 职工号→部门号
- 职工和产品之间多对多的联系可表示为
  - （职工号，产品号）→工作天数
- 得到函数依赖集$F_L$

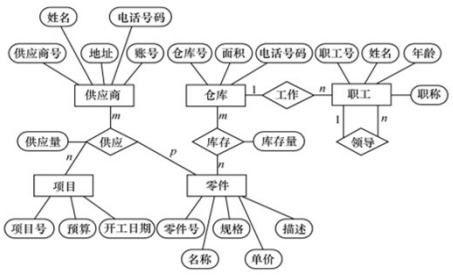
## 消除冗余的方法（7）

- 方法（续）

  2. 求$F_L$的最小覆盖$G_L$，差集为$D = F_L - G_L$。逐一考察D中的函数依赖，确定是否是冗余的联系，若是，就把它去掉
  - 冗余的联系一定在D中，而D中的联系不一定是冗余的
  - 当实体之间存在多种联系时要将实体之间的联系在形式上加以区分

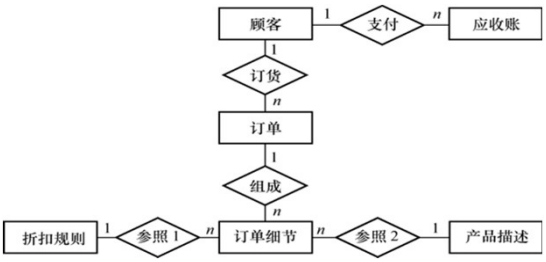
## 实例（1）

- 某工厂管理信息系统的视图集成
  – 工厂物资管理E-R图



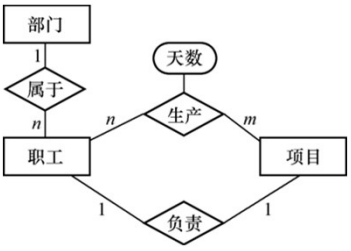

## 实例（2）

– 销售管理子系统的分E-R图



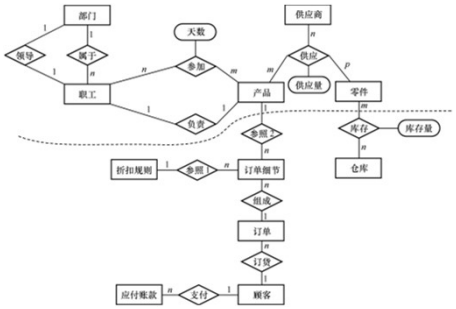

## 实例（3）

– 劳动人事管理的分E-R图




## 实例（4）

- 某工厂管理信息系统的基本E-R图



## 实例（5）

- 集成过程，解决了以下问题
  - 异名同义，项目和产品含义相同
  - 库存管理中职工与仓库的工作关系已包含在劳动人事管理的部门与职工之间的联系之中，所以可以取消
  - 职工之间领导与被领导关系可由部门与职工（经理）之间的领导关系、部门与职工之间的从属关系两者导出，所以也可以取消

## 验证整体概念结构（1）

- 视图集成后形成一个整体的数据库概念结构，对该整体概念结构还必须进行进一步验证，确保它能够满足下列条件
  - 整体概念结构内部必须具有一致性，不存在互相矛盾的表达
  - 整体概念结构能准确地反映原来的每个视图结构，包括属性、实体及实体间的联系
  - 整体概念结构能满足需求分析阶段所确定的所有要求

## 验证整体概念结构（2）

- 整体概念结构最终还应该提交给用户，征求用户和有关人员的意见，进行评审、修改和优化，然后把它确定下来，作为数据库的概念结构，作为进一步设计数据库的依据

## 5.3.5 概念结构设计小结（1）

- 概念结构设计的步骤
  - 抽象数据并设计局部视图
  - 集成局部视图，得到全局概念结构
  - 验证整体概念结构
- 数据抽象
  - 分类
  - 聚集
  - 概括

## 5.3.5 概念结构设计小结（2）

- 设计局部视图
  1. 选择局部应用
  2. 逐一设计分E-R图
     - 标定局部应用中的实体、属性、键，实体间的联系
     - 用E-R图描述出来

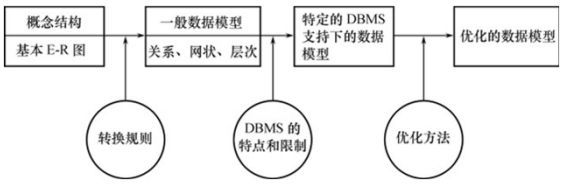
## 5.3.5 概念结构设计小结（3）

- 集成局部视图
  1. 合并分E-R图，生成初步E-R图
     - 消除冲突
       - 属性冲突
       - 命名冲突
       - 结构冲突
  2. 修改与重构
     - 消除不必要的冗余，设计生成基本E-R图
       - 分析方法
       - 规范化理论

## 5.4 逻辑结构设计（1）

- 逻辑结构设计的任务
  - 把概念结构设计阶段设计好的基本E-R图转换为与选用DBMS产品所支持的数据模型相符合的逻辑结构
- 逻辑结构设计的步骤
  - 将概念结构转化为一般的关系、网状、层次模型
  - 将转换来的关系、网状、层次模型向特定DBMS支持下的数据模型转换
  - 对数据模型进行优化

## 5.4 逻辑结构设计（2）




## 5.4 逻辑结构设计（3）

- 5.4.1 E-R图向关系模型的转换
- 5.4.2 数据模型的优化
- 5.4.3 设计用户子模式
- 5.4.4 逻辑结构设计小结

## 5.4.1 E-R图向关系模型的转换（1）

- 转换内容
- 转换原则

## 5.4.1 E-R图向关系模型的转换（2）

- E-R图向关系模型的转换要解决的问题
  - 如何将实体型和实体间的联系转换为关系模式
  - 如何确定这些关系模式的属性和键
- 转换内容
  - 将E-R图转换为关系模型：将实体、实体的属性和实体之间的联系转换为关系模式

## 5.4.1 E-R图向关系模型的转换（3）

- 实体型间的联系有以下不同情况

  （1）一个1:1联系可以转换为一个独立的关系模式，也可以与任意一端对应的关系模式合并
  - 转换为一个独立的关系模式
  - 与某一端实体对应的关系模式合并

  （2）一个1:n联系可以转换为一个独立的关系模式，也可以与n端对应的关系模式合并
  - 转换为一个独立的关系模式
  - 与n端对应的关系模式合并

## 5.4.1 E-R图向关系模型的转换（4）

- 实体型间的联系有以下不同情况（续）
  （3）一个m:n联系转换为一个关系模式
  - 例，“选修”联系是一个m:n联系，可以将它转换为如下关系模式，其中学号与课程号为关系的组合键
    选修（学号，课程号，成绩）

## 5.4.1 E-R图向关系模型的转换（5）

- 实体型间的联系有以下不同情况（续）
  （4）三个或三个以上实体间的一个多元联系转换为一个关系模式
  - 例，“讲授”联系是一个三元联系，可以将它转换为如下关系模式，其中课程号、职工号和书号为关系的组合键
    讲授（课程号，职工号，书号）

## 5.4.1 E-R图向关系模型的转换（6）

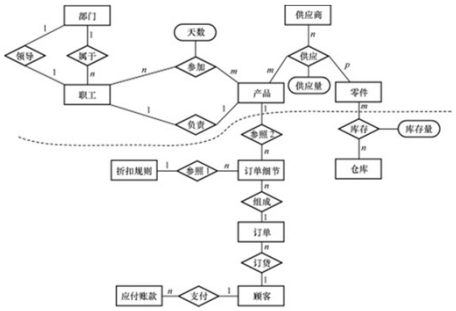
- 实体型间的联系有以下不同情况（续）
  （5）具有相同键的关系模式可合并
  - 目的：减少系统中的关系个数
  - 合并方法：将其中一个关系模式的全部属性加入到另一个关系模式中，然后去掉其中的同义属性（可能同名也可能不同名），并适当调整属性的次序

## 5.4.1 E-R图向关系模型的转换（7）

- 注意
  - 从理论上讲，1:1联系可以与任意一端对应的关系模式合并
  - 但在一些情况下，与不同的关系模式合并效率会大不一样。因此究竟应该与哪端的关系模式合并需要依应用的具体情况而定
  - 由于连接操作是最费时的操作，所以一般应以尽量减少连接操作为目标
    - 例如，如果经常要查询某个班级的班主任姓名，则将管理联系与教师关系合并更好些

## 5.4.1 E-R图向关系模型的转换（8）

- 某工厂管理信息系统的 基本E-R图




## 5.4.1 E-R图向关系模型的转换（9）

- 把上图中虚线上部的E-R图转换为关系模型
  - 部门实体对应的关系模式
    部门（部门号，部门名，经理的职工号，…）
    - 此关系模式已包含了联系“领导”所对应的关系模式
    - 经理的职工号是关系的候选键
  - 职工实体对应的关系模式
    职工（职工号，部门号，职工名，职务，…）
    - 该关系模式已包含了联系“属于”所对应的关系模式

## 5.4.1 E-R图向关系模型的转换（10）

- 把上图中虚线上部的E-R图转换为关系模型（续）
  - 产品实体对应的关系模式
    产品（产品号，产品名，产品组长的职工号，…）
  - 供应商实体对应的关系模式
    供应商（供应商号，姓名，…）
  - 零件实体对应的关系模式
    零件（零件号，零件名，…）

## 5.4.1 E-R图向关系模型的转换（11）

- 把上图中虚线上部的E-R图转换为关系模型（续）
  - 联系“参加”所对应的关系模式
    职工工作（职工号，产品号，工作天数，…）
  - 联系“供应”所对应的关系模式
    供应（产品号，供应商号，零件号，供应量）

## 5.4.2 数据模型的优化

- 得到初步数据模型后，还应该适当地修改、调整数据模型的结构，以进一步提高数据库应用系统的性能，这就是数据模型的优化
- 关系数据模型的优化通常以规范化理论为指导

## 优化数据模型的方法（1）

（1）确定数据依赖
- 按需求分析阶段所得到的语义，分别写出每个关系模式内部各属性之间的数据依赖以及不同关系模式属性之间数据依赖

（2）消除冗余的联系
- 对于各个关系模式之间的数据依赖进行极小化处理，消除 冗余的联系

## 优化数据模型的方法（2）

（3）确定所属范式
- 按照数据依赖的理论对关系模式逐一进行分析
- 考查是否存在部分函数依赖、传递函数依赖、多值依赖等
- 确定各关系模式分别属于第几范式

（4）按照需求分析阶段得到的各种应用对数据处理的要求，分析对于这样的应用环境这些模式是否合适，确定是否要对它们进行合并或分解

## 优化数据模型的方法（3）

- 注意：并不是规范化程度越高的关系就越优，一般说来，第三范式就足够了
  - 例：在关系模式学生成绩单（学号，英语，数学，语文，平均成绩）中存在下列函数依赖
    学号→英语
    学号→数学
    学号→语文
    学号→平均成绩
    （英语, 数学, 语文）→平均成绩

## 优化数据模型的方法（4）

显然有

学号→（英语，数学，语文）

因此该关系模式中存在传递函数依赖，是2NF关系

虽然平均成绩可以由其他属性推算出来，但如果应用中需要经常查询学生的平均成绩，为提高效率，仍然可保留该冗余数据，对关系模式不再做进一步分解

## 优化数据模型的方法（5）

（5）按照需求分析阶段得到的各种应用对数据处理的要求，对关系模式进行必要的分解，以提高数据操作的效率和存储空间的利用率

- 常用分解方法
  - 水平分解
  - 垂直分解

## 优化数据模型的方法（6）

- 水平分解
  - 什么是水平分解
    - 把（基本）关系的元组分为若干子集合，定义每个子集合为一个子关系，以提高系统的效率
  - 水平分解的适用范围
    - 满足“80/20原则”的应用
    - 并发事务经常存取不相交的数据

## 优化数据模型的方法（7）

- 垂直分解
  - 什么是垂直分解
    - 把关系模式R的属性分解为若干子集合，形成若干子关系模式
  - 垂直分解的适用范围
    - 取决于分解后R上的所有事务的总效率是否得到了提高

## 5.4.3 设计用户子模式（1）

- 定义用户外模式时应该注重的问题，包括三个方面
  - 使用更符合用户习惯的别名
  - 针对不同级别的用户定义不同的View，以满足系统对安全性的要求
  - 简化用户对系统的使用

## 5.4.3 设计用户子模式（2）

- 例：关系模式产品（产品号，产品名，规格，单价，生产车间，生产负责人，产品成本，产品合格率，质量等级），可以在产品关系上建立两个视图
  - 为一般顾客建立视图
    - 产品1（产品号，产品名，规格，单价）
  - 为产品销售部门建立视图
    - 产品2（产品号，产品名，规格，单价，车间，生产负责人）

## 5.4.3 设计用户子模式（3）

- 顾客视图中只包含允许顾客查询的属性
- 销售部门视图中只包含允许销售部门查询的属性
- 生产领导部门则可以查询全部产品数据
- 可以防止用户非法访问不允许他们查询的数据，保证系统的安全性

## 5.4.4 逻辑结构设计小结（1）

- 任务
  - 将概念结构转化为具体的数据模型
- 逻辑结构设计的步骤
  - 将概念结构转化为一般的关系、网状、层次模型
  - 将转化来的关系、网状、层次模型向特定DBMS支持下的数据模型转换
  - 对数据模型进行优化
  - 设计用户子模式

## 5.4.4 逻辑结构设计小结（2）

- 优化数据模型的方法
  1. 确定数据依赖
  2. 对于各个关系模式之间的数据依赖进行极小化处理，消除冗余的联系
  3. 确定各关系模式分别属于第几范式
  4. 分析对于应用环境这些模式是否合适，确定是否要对它们进行合并或分解
  5. 对关系模式进行必要的分解或合并

## 5.4.4 逻辑结构设计小结（3）

- 设计用户子模式
  1. 使用更符合用户习惯的别名
  2. 针对不同级别的用户定义不同的外模式，以满足系统对安全性的要求
  3. 简化用户对系统的使用

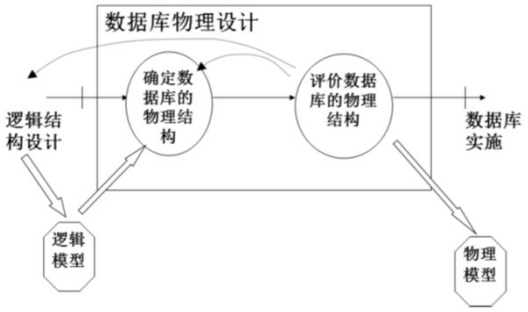
## 5.5 数据库的物理设计（1）

- 数据库的物理设计
  - 数据库在物理设备上的存储结构与存取方法称为数据库的物理结构，它依赖于选定的数据库管理系统
  - 为一个给定的逻辑数据模型选取一个最适合应用环境的物理结构的过程，就是数据库的物理设计

## 5.5 数据库的物理设计（2）

- 数据库物理设计的步骤
  - 确定数据库的物理结构，在关系数据库中主要指存取方法和存储结构
  - 对物理结构进行评价，评价的重点是时间和空间效率
    - 如果评价结果满足原设计要求，则可进入到物理实施阶段，否则，就需要重新设计或修改物理结构，有时甚至要返回逻辑设计阶段修改数据模型

## 5.5 数据库的物理设计（3）




## 5.5 数据库的物理设计（4）

- 5.5.1 数据库物理设计的内容和方法
- 5.5.2 关系模式存取方法选择
- 5.5.3 确定数据库的存储结构
- 5.5.4 评价物理结构

## 5.5.1 数据库物理设计的内容和方法（1）

- 设计物理数据库结构的准备工作
  - 对要运行的事务进行详细分析，获得选择物理数据库设计所需参数
  - 充分了解所用RDBMS的内部特征，特别是系统提供的存取方法和存储结构

## 5.5.1 数据库物理设计的内容和方法（2）

- 选择物理数据库设计所需参数
  - 数据库查询事务
    - 查询的关系
    - 查询条件所涉及的属性
    - 连接条件所涉及的属性
    - 查询的投影属性
  - 数据更新事务
    - 被更新的关系
    - 每个关系上的更新操作条件所涉及的属性
    - 修改操作要改变的属性值
  - 每个事务在各关系上运行的频率和性能要求

## 5.5.1 数据库物理设计的内容和方法（3）

- 关系数据库物理设计的内容
  - 为关系模式选择存取方法（建立存取路径）
  - 设计关系、索引等数据库文件的物理存储结构

## 5.5.2 关系模式存取方法选择（1）

- 数据库系统是多用户共享的系统，对同一个关系要建立多条存取路径才能满足多用户的多种应用要求
- 物理设计的任务之一就是要确定选择哪些存取方法，即建立哪些存取路径

## 5.5.2 关系模式存取方法选择（2）

- DBMS常用存取方法
  - 索引方法
    - 目前主要是B+树索引方法
    - 经典存取方法，使用最普遍
  - 聚簇（Cluster）方法
  - HASH方法

## 索引方法（1）

- 基本概念
  - 索引域：数据文件中定义索引的一个或一组域
  - 索引记录/索引项：索引文件的记录，包括两个域
    - 索引域：存储索引域的一个值（K）
    - 指针：指向索引域值为K的记录所在磁盘块的地址

## 索引方法（2）

- 分类
  - 按照索引文件结构
    - 稀疏索引：把所有数据记录按关键字值分成许多组，每组设立一个索引项
    - 稠密索引：为每个记录设立一个索引项，记录存放是任意的，但索引是有序的

## 多级索引（1）

- 二叉树索引
  - 每个节点只有一个关键字，有两个指针，分别指向关键字值小于或大于当前节点值的节点
- B树（平衡树）索引
  - 是附加限制条件的索引树。限制了每个节点放置关键字与指针的最小和最大个数，并且所有的叶节点都在同一层上。B树的关键字是散布在各层上

## 多级索引（2）

- B+树索引
  - 是B树的改进。把树中所有关键字都按递增次序从左到右安排在叶节点上，并且链接起来。$B^+$树能同时进行随机查找和顺序查找

## 索引存取方法的选择

- 选择索引域原则
  - 经常在查询条件中出现的属性
  - 经常作为最大值和最小值库函数的参数
  - 经常作为连接属性
- 索引并非越多越好

## 聚簇方法

- 基本概念
  - 把某个属性/组（聚簇键）值相同的关系的记录集中存放在连续的物理块，称为聚簇。能够提高该属性的查询速度
  - 把经常进行连接操作的关系的记录以连接属性为中心进行存储，连接属性为聚簇键。提高连接操作的效率
- 一个关系只能参加一个聚簇

## 聚簇存取方法的选择

- 一般原则
  - 经常进行连接操作的关系可建立聚簇
  - 单个关系的某组属性经常进行相等比较
  - 关系的某个属性组值重复率高
- 注意问题
  - 建立与维护聚簇系统开销很大，对于更新操作远远多于连接操作的关系不应使用聚簇方法

## HASH文件

- 一种支持快速存取的文件存储方法
- 基本概念
  - 通过HASH函数将记录关键字转换成地址。
  - 设F是一个文件，A是F的HASH域，H是定义在A的值域上的HASH函数。对于记录r（其A的值为a）的存储地址由H（r）确定
- 动态HASH方法

## HASH存取方法的选择

- 如果关系的属性主要出现在等值连接条件中，或出现在相等比较条件中，而且满足下列条件之一，可以选择该方法
  - 关系的大小可预知，而且不变
  - 如果关系大小动态改变，则须DBMS提供动态HASH存取方法

## 5.5.3 确定数据库的存储结构

- 确定数据库物理结构的内容
  1. 确定数据的存放位置和存储结构
     - 关系
     - 索引
     - 聚簇
     - 日志
     - 备份
  2. 确定系统配置

## 1. 确定数据的存放位置（1）

- 确定数据存放位置和存储结构的因素
  - 存取时间
  - 存储空间利用率
  - 维护代价

这三个方面常常是相互矛盾的

- 例：消除一切冗余数据虽能够节约存储空间和减少维护代价，但往往会导致检索代价的增加
- 必须进行权衡，选择一个折中方案

## 1. 确定数据的存放位置（2）

- 基本原则
  - 根据应用情况将
    - 易变部分与稳定部分分开存放
    - 存取频率较高部分与存取频率较低部分，分开存放

## 1. 确定数据的存放位置（3）

- 基本原则（续）
  - 例
    - 数据库数据备份、日志文件备份等由于只在故障恢复时才使用，而且数据量很大，可以考虑存放在磁带上
    - 如果计算机有多个磁盘或磁盘阵列，可以考虑将表和索引分别放在不同的磁盘上，在查询时，由于磁盘驱动器并行工作，可以提高物理I/O读写的效率
    - 可以将比较大的表分别放在两个磁盘上，以加快存取速度，这在多用户环境下特别有效
    - 可以将日志文件与数据库对象（表、索引等）放在不同的磁盘以改进系统的性能

## 2. 确定系统配置

- DBMS产品一般都提供了一些存储分配参数
  - 同时使用数据库的用户数
  - 同时打开的数据库对象数
  - 内存分配参数
  - 使用的缓冲区长度、个数
  - 存储分配参数
  - …….

## 5.5.4 评价物理结构（1）

- 评价内容
  - 对数据库物理设计过程中产生的多种方案进行细致的评价，从中选择一个较优的方案作为数据库的物理结构

## 5.5.4 评价物理结构（2）

- 评价方法（完全依赖于所选用的DBMS ）
  - 定量估算各种方案
    - 存储空间
    - 存取时间
    - 维护代价
  - 对估算结果进行权衡、比较，选择出一个较优的合理的物理结构
  - 如果该结构不符合用户需求，则需要修改设计

## 5.6 数据库实施和维护

- 5.6.1 数据的载入和应用程序的调试
- 5.6.2 数据库的试运行
- 5.6.3 数据库的运行和维护

## 5.6.1 数据的载入和应用程序的调试

- 数据的载入
- 应用程序的编码和调试

## 数据的载入

- 数据库结构建立好后，就可以向数据库中装载数据了。组织数据入库是数据库实施阶段最主要的工作
- 数据装载方法
  - 人工方法
  - 计算机辅助数据入库

## 应用程序的编码和调试

- 数据库应用程序的设计应该与数据设计并行进行
- 在组织数据入库的同时还要调试应用程序

## 5.6.2 数据库的试运行（1）

- 在原有系统的数据有一小部分已输入数据库后，就可以开始对数据库系统进行联合调试，称为数据库的试运行
- 数据库试运行主要工作包括
  1）功能测试
  - 实际运行数据库应用程序，执行对数据库的各种操作，测试应用程序的功能是否满足设计要求
  - 如果不满足，对应用程序部分则要修改、调整，直到达到设计要求

## 5.6.2 数据库的试运行（2）

- 数据库试运行主要工作包括（续）
  2）性能测试
  - 测量系统的性能指标，分析是否达到设计目标
  - 如果测试的结果与设计目标不符，则要返回物理设计阶段，重新调整物理结构，修改系统参数，某些情况下甚至要返回逻辑设计阶段，修改逻辑结构

## 5.6.2 数据库的试运行（3）

- 强调两点
  - 分期分批组织数据入库
    - 重新设计物理结构甚至逻辑结构，会导致数据重新入库
    - 由于数据入库工作量实在太大，费时、费力，所以应分期分批地组织数据入库
    - 先输入小批量数据供调试用
      - 待试运行基本合格后再大批量输入数据
      - 逐步增加数据量，逐步完成运行评价

## 5.6.2 数据库的试运行（4）

- 强调两点（续）
  - 数据库的转储和恢复
    - 在数据库试运行阶段，系统还不稳定，硬、软件故障随时都可能发生
    - 系统的操作人员对新系统还不熟悉，误操作也不可避免
    - 因此必须做好数据库的转储和恢复工作，尽量减少对数据库的破坏

## 5.6.3 数据库的运行和维护 （1）

- 在数据库运行阶段，对数据库经常性的维护工作主要是由DBA完成的，包括
  - 数据库的转储和恢复
  - 数据库的安全性、完整性控制
  - 数据库性能的监督、分析和改进
  - 数据库的重组织和重构造

## 5.6.3 数据库的运行和维护 （2）

- 数据库的重组织和重构造
  - 重组织的形式
    - 全部重组织
    - 部分重组织
      - 只对频繁增、删的表进行重组织
  - 重组织的目标
    - 提高系统性能

## 5.6.3 数据库的运行和维护 （3）

- 数据库的重组织和重构造（续）
  - 重组织的工作
    - 按原设计要求
      - 重新安排存储位置
      - 回收垃圾
      - 减少指针链
    - 数据库的重组织不会改变原设计的数据逻辑结构和物理结构

## 5.6.3 数据库的运行和维护 （4）

- 数据库的重组织和重构造（续）
  - 数据库重构造
    - 根据新环境调整数据库的模式和内模式
      - 增加新的数据项
      - 改变数据项的类型
      - 改变数据库的容量
      - 增加或删除索引
      - 修改完整性约束条件

## 5.7 IDEF1X数据建模

- 5.7.1 IDEF1X数据建模方法
- 5.7.2 IDEF1X的语法和语义
- 5.7.3 IDEF1X建模过程

## 5.7.1 IDEF1X数据建模方法（1）

- IDEF分析方法统称为IDEF（Integration DEFinition）
  - 是70年代中期美国空军的ICAM（Integration Computer Aided Manufacturing）计划下所发展出来的一套系统分析与设计的工具
  - ICAM计划的宗旨是通过系统地应用计算机技术来提高机械制造业的生产率
  - 本来只是运用在制造业上的，经过改造后用途变广泛了，适用于一般的软件开发

## 5.7.1 IDEF1X数据建模方法（2）

- 从IDEF中的每套方法都是通过建模程序来获取某个特定类型的信息
  - IDEF0 功能建模（Function Modeling）
  - IDEF1 信息建模（Information Modeling）
  - IDEF1X 数据建模（Data Modeling）
  - IDEF2 仿真建模设计（Simulation Modeling Design）
  - IDEF3 过程描述获取（Process Description Capture）

## 5.7.1 IDEF1X数据建模方法（3）

- 1983年美国空军ICAM计划着手IISS项目
  - 目的是提供一套能在逻辑上和物理上集成异种计算机硬件和软件的技术
  - IISS把集成的途径放在数据源单一语义定义的获取、管理和使用上，这个“概念模型” 是用IDEF1建模技术来定义的

## 5.7.1 IDEF1X数据建模方法（4）

- IDEF1X（Data Modeling）把实体-联系方法应用到语义数据模型中
- IDEF1X对IDEF1做了一些改进，增强了图形表达能力,丰富了语义（如分类联系的引入）和简化了开发过程
- 实际上IDEF1X是IDEF1的延伸版本，其之间的差异主要在于IDEF1是用来描述系统中数据的储存及数据之间的关系，而IDEF1X着重于实体属性和实体关系之间的数据模型

## 5.7.1 IDEF1X数据建模方法（5）

- IDEF1X在数据建模方法具有如下的特点
  - （1）支持概念模式的开发
  - （2）是一种一致性的语言
  - （3）IDEF1X是可教授的
  - （4）经过良好的测试和验证
  - （5）可以自动化

## 5.7.2 IDEF1X的语法和语义

- 实体
- 属性
- 主键
- 确定性联系
- 标识联系与非标识联系
- 分类联系
- 非确定联系（多对多联系）
- 外键
- 角色

## 实体（1）

- 实体（entity）是现实世界中具有一组相同属性的“事务”或“对象”的集合，集合中的每个元素称为实体的实例（entity instance）
- 实体有两种类型
  - 独立标识实体（Identifier-Independent Entities），简称独立实体
  - 从属标识实体（Identifier-Dependent Entities），简称从属实体

## 实体（2）

- 独立实体
- 独立实体与从属实体

## 实体（3）

- IDEF1X关于实体规则定义如下
  - 每一个实体必须使用唯一的实体名
  - 一个实体可以有一个或多个属性，这些属性可以是实体自身所具有的，也可以是通过一个联系而继承得到的
  - 一个实体应有一个或多个能唯一标识实体中每一个实例的属性（主关键字和次关键字）

## 实体（4）

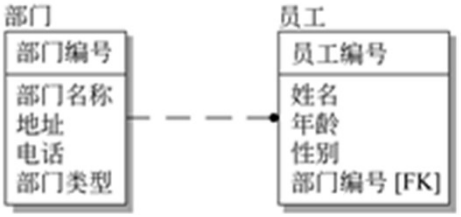
- IDEF1X关于实体规则定义如下（续）
  - 任意实体都可以与模型中其它的实体有0个、1个或多个联系
  - 如果一个外键是一个主键的全部或部分，那么该实体就是从属实体。如果根本没有外键属性用作一个实体的主键，那么这个实体就是独立实体

## 属性

- 属性是与具体的实体相联系的
- 实体的每一个属性必须具有一个单一且确定的值

## 主键

- 一个实体必须具有一个属性或属性组，其值唯一地确定该实体中的每一个实例



## 确定性联系（1）

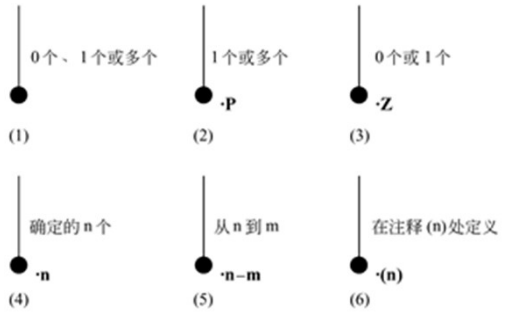
- 确定性联系（specific connection relationship）或简称联系（relationship）是指实体间的相互关联
  - 父实体的每一个实例都与子实体的0个、1个或多个实例相关联
  - 子实体中的1个实例与父实体中的0或1个实例相关联

## 确定性联系（2）

- 联系有三个属性
  - 名称
  - 基数（cardinality，有时也称为度量）
  - 联系的类型（标识或者非标识、非空）

## 确定性联系（3）

- IDEF1X中的联系类型

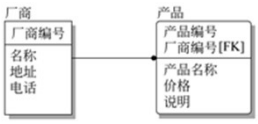



## 标识联系与非标识联系（1）

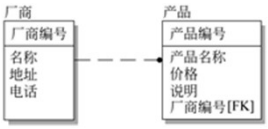
- 确定性联系可以划分为
  - 标识联系
    - 子实体的一个实例通过父实体标识，即子实体依赖于父实体而不能单独存在
    - 也可以解释为父实体的主键在子实体中既是外键又是主键的一部分或全部
  - 非标识联系
    - 父实体中的主键在子实体中仅作为外键

## 标识联系与非标识联系（2）

- 标识联系




- 非标识联系

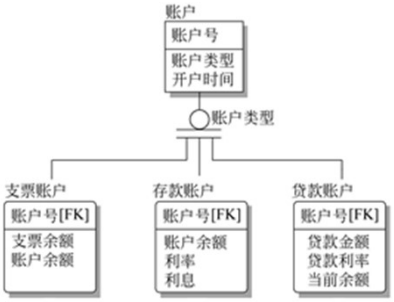



## 分类联系（1）

- 当多个实体有共同的属性时，而这些实体又分别有一些不同的属性时，可以创建一个一般实体并将这些实体共同的属性作为一般实体的属性
  - 如银行应用中的储蓄账号、支票账号和贷款账号，其中都有开户人姓名、开户时间这两个属性
  - 这些实体又都各自有一些特殊的属性，在数据建模过程中可以使用分类联系进行处理

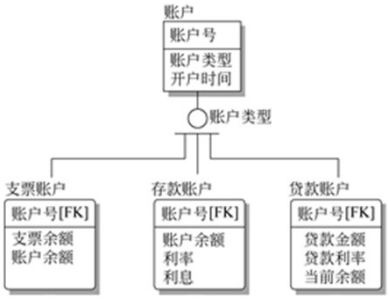
## 分类联系（2）

- 完全分类联系




## 分类联系（3）

- 不完全分类联系




## 非确定联系（多对多联系）（1）

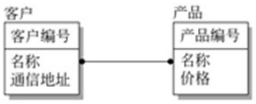
- 在完善的IDEF1X模型中，实体间的所有联系都必须用确定联系（包括分类联系）来描述。但是，在建模的初级阶段，只需要发现实体间有没有联系即可，至于到底是哪种类型的联系并不重要

## 非确定联系（多对多联系）（2）

- 只要通过分析知道两个实体间存在联系，就可以将其描述为“非确定联系”
  - 两个实体间的非确定联系通常称为多对多联系
  - 多对多联系常常隐藏了某些业务规则， 应当在随后的建模过程中进一步细化。通常的做法是
    - 引入一个新的实体作为中间联系实体，将一个多对多的联系分解成两个一对多的联系

## 非确定联系（多对多联系）（3）

- 多对多联系



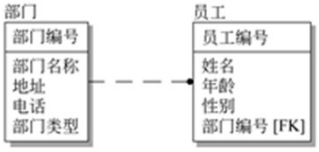

- 多对多联系的分解

## 外键

- 如果在两个实体之间存在确定联系或分类联系，那么构成父实体或一般实体主键的属性被迁移到子实体或分类实体的属性，这些迁移属性被称为“外键（foreign key）”

## 角色（1）

- 实体在联系中的作用称为实体的角色（role），它是通过迁移属性体现的
- 由于参与联系的实体通常是互异的，角色是隐含的并且不需要指定




## 角色（2）

- 一个实体作为父实体与同一个子实体有多个联系的时候，则父实体每次参与联系都有不同的角色
- 例如，“员工”实体与“合同”实体有两个联系
  - 迁移属性“员工编号”的作用是“销售”
  - 迁移属性“员工编号”的作用是“技术审核”
  - 在IDEF1X的表示中，必须为迁移属性的角色命名且角色名是唯一的

## 角色（3）

- 父实体和子实体有多个联系的角色

## 角色（4）

- 递归联系（实体到实体自身的联系），定义角色就是给外键属性重新命名，如果没有定义角色，外键属性的名称就是父实体中的主键的属性名
  - 例如，“员工”实体中的“部门经理”是一个角色名，它是该员工的部门经理的员工编号

## 5.7.3 IDEF1X建模过程

- 阶段0——设计的开始
- 阶段1——定义实体
- 阶段2——定义联系
- 阶段3——定义键
- 阶段4——定义属性

## 5.8 小结（1）

- 数据库的设计过程
  - 需求分析
  - 概念结构设计
  - 逻辑结构设计
  - 物理设计
  - 实施和维护

## 5.8 小结（2）

- 数据库各级模式的形成
  - 数据库的各级模式是在设计过程中逐步形成的
  - 需求分析阶段综合各个用户的应用需求（现实世界的需求）
  - 概念设计阶段形成独立于机器特点、独立于各个DBMS产品的概念模式（信息世界模型），用E-R图来描述

## 5.8 小结（3）

- 数据库各级模式的形成（续）
  - 在逻辑设计阶段将E-R图转换成具体的数据库产品支持的数据模型如关系模型，形成数据库逻辑模式。然后根据用户处理的要求，安全性的考虑，在基本表的基础上再建立必要的视图（View）形成数据的外模式
  - 在物理设计阶段根据DBMS特点和处理的需要，进行物理存储安排，设计索引，形成数据库内模式

## 5.8 小结（4）

- 数据库设计包含的艺术成分多于科学成分。尽管在数据库设计领域中有一些理论或方法可以应用，但仅靠这些理论或方法并不能很好地完成数据库设计，它需要大量的经验和创造性

## 参考资料


- 第七章，王珊 萨师煊，数据库系统概论（第5版）， 北京：高等教育出版社，2014
- 第11章，周志逵 郭贵锁等编著，数据库系统原理，北京：清华大学出版社，2008
- 第六章，萨师煊 王珊，数据库系统概论（第三版）， 北京：高等教育出版社，2000


Thank You